



Entity-Relation Joint Extraction Method Based on Reinforcement Learning and Global Pointer Network

Weidong Pan^{*}, Yijie Li

College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, 210007 Nanjing, China

^{*} Correspondence: Weidong Pan (1sz2305187@nuaa.edu.cn)

Received: 06-12-2025

Revised: 07-28-2025

Accepted: 08-23-2025

Citation: W. D. Pan and Y. J. Li, "Entity-relation joint extraction method based on reinforcement learning and global pointer network," *Int J. Knowl. Innov Stud.*, vol. 3, no. 3, pp. 158–177, 2025. <https://doi.org/10.56578/ijkis030303>.



© 2025 by the authors. Licensee Acadlore Publishing Services Limited, Hong Kong. This article can be downloaded for free, and reused and quoted with a citation of the original published version, under the CC BY 4.0 license.

Abstract: Entity–relation extraction constitutes a fundamental step in the construction of domain-specific knowledge graphs. In fault analysis of transmission systems, this task is complicated by extensive entity–relation overlap, nested structures, and strong semantic dependencies in technical texts. To address these challenges, an entity–relation joint extraction framework integrating reinforcement learning with a global pointer network (GPN) is developed (joint extraction model based on GPN and reinforcement learning, RL-BGPNet). A fault-oriented dataset is first established from helicopter transmission system maintenance manuals and related technical documents. Global semantic associations are then captured through a relation-aware attention mechanism, while parallel decoding is achieved using a GPN to accommodate overlapping and nested entities. The extraction of entity–relation triplets is further formulated as a multi-step decision process under a reinforcement learning paradigm, enabling coordinated optimization of entity recognition and relation classification and alleviating error accumulation caused by task interference. Experimental evaluations demonstrate that the proposed framework maintains stable performance under complex semantic conditions and exhibits satisfactory generalization, supporting its application to knowledge extraction and preliminary knowledge graph construction in the helicopter transmission system fault domain.

Keywords: Knowledge graph; Entity–relation extraction; Reinforcement learning; Global pointer network

1 Introduction

A knowledge graph (KG) is essentially a semantic network whose fundamental structure is composed of triplets in the form of (head entity, relation, tail entity). It was formally introduced by Google in 2012 [1] and has since become a core paradigm for organizing and representing structured knowledge. As an effective bridge between artificial intelligence and big data technologies [2], KG construction aims to transform large volumes of unstructured textual information into structured knowledge, primarily through entity recognition and relation extraction. By enabling large-scale semantic networks [3], KGs facilitate deep associations among heterogeneous data and support graph-based traversal and reasoning, which has led to extensive applications across a wide range of domains [4–6].

In recent years, neural-network-based approaches have achieved substantial progress in named entity recognition (NER). The BiLSTM–CRF architecture proposed by Ammaduddin et al. [7] has become a canonical baseline, establishing a standard paradigm that separates encoding and decoding stages. With the advent of pretrained language models, Devlin et al. [8] introduced bidirectional Transformer representations that provide dynamic and context-sensitive semantic features, effectively addressing lexical ambiguity. Nevertheless, sequence-labeling-based methods generally assume that each token belongs to a single entity category, which limits their ability to handle nested or overlapping entities and results in inherent deficiencies when confronted with complex entity structures [9]. To overcome this limitation, Yu et al. [10] reformulated NER as an edge prediction problem on dependency graphs using a biaffine attention mechanism, thereby extending the modeling capacity beyond the constraints of conventional sequence labeling.

Early research on relation extraction was largely dominated by pipeline architectures [11], in which entities are first identified and relations are subsequently classified. To mitigate the high annotation cost associated with large-scale datasets, Zeng et al. [12] proposed a piecewise convolutional neural network (PCNN) combined with multi-instance learning, which effectively reduced noise in distantly supervised data. However, the inherent error

propagation problem of pipeline methods motivated a transition toward joint extraction frameworks. Miwa and Bansal [13] introduced an end-to-end stacked LSTM model with shared parameters to enable interaction between entity and relation features. Subsequently, Hu et al. [14] proposed the CasRel cascade framework, which adopts a hierarchical strategy by first detecting subjects and then identifying relation-specific objects, achieving improved performance on entity overlap. Despite these advances, the discrepancy between training with gold subjects and inference with predicted subjects leads to exposure bias in cascade decoding. To address this issue, Wang et al. [15] developed the TPLinker model, which employs a “handshaking” tagging scheme to construct token-pair matrices and enables single-stage, bias-free extraction. However, the resulting matrix grows quadratically with sequence length, $O(N^2)$, introducing severe sparsity and memory overhead when processing long documents.

In addition to these general challenges, fault-related texts in helicopter transmission systems exhibit distinct engineering characteristics that further complicate entity–relation extraction. First, long composite entities are pervasive: fault components such as “the driving bevel gear at the input end of the tail gearbox” often exceed ten characters, making them prone to semantic fragmentation under conventional methods. Second, triplet density is high: an average sentence contains approximately 5.4 triplets, and one-to-many relations are common, for instance when a single fault mode gives rise to multiple observable phenomena, which increases the likelihood of relation confusion. Third, domain-specific terminology tends to have fixed semantics but ambiguous boundaries: entities such as fault modes and maintenance measures rely heavily on expert knowledge and are difficult to distinguish solely from local textual context.

To address these challenges, an entity–relation joint extraction framework based on reinforcement learning [16] and a global pointer network (GPN) [17] is developed. The proposed approach adopts a three-level architecture characterized by type guidance, parallel decoding, and global optimization. Entity type cross-attention is first introduced to incorporate domain-specific entity types from helicopter transmission systems as prior knowledge into semantic encoding, thereby reducing semantic ambiguity when the same token participates in multiple roles. A GPN is then employed to support unbiased and parallel extraction of nested entities, overcoming the boundary constraints inherent to sequence-labeling approaches. Finally, reinforcement learning is used to directly optimize the F1 score as a reward signal, forming a closed-loop process of prediction, feedback, and parameter updating, and resolving the inconsistency between local loss optimization and global performance objectives that is common in conventional supervised learning.

The remainder of this paper is organized as follows. Section 2 describes the construction of the helicopter transmission system fault text dataset. Section 3 presents the decoding architecture based on the GPN. Section 4 introduces the reinforcement-learning-based decoding optimization strategy. Section 5 illustrates the overall entity–relation extraction model integrating reinforcement learning and the GPN, together with the workflow for KG construction. Section 6 reports experimental results on both public benchmark datasets and the helicopter transmission system fault dataset. Finally, conclusions are drawn, and the overall research framework is shown in Figure 1.

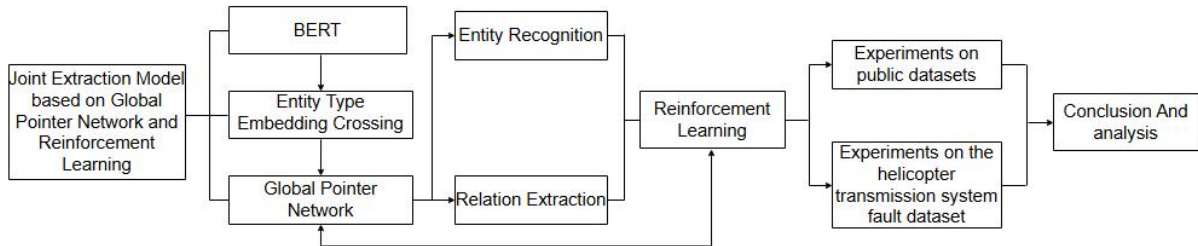


Figure 1. Research framework

2 Construction of the Fault Text Dataset

2.1 Data Sources and Ontology Definition

Prior to knowledge extraction for helicopter transmission system faults, information related to transmission system failures was collected from multiple domain-specific sources, including literature surveys, maintenance manuals, and patent documents. Fault descriptions relevant to helicopter transmission systems were extracted from these materials and subsequently subjected to data cleaning and filtering procedures to construct an entity–relation dataset.

To achieve structured knowledge representation, annotation guidelines were established in consultation with senior experts in helicopter maintenance engineering. Based on these guidelines, five categories of key entities were defined, forming the core ontology of the fault analysis KG for transmission system components, as summarized in Table 1.

In this study, relationships between entities were categorized into five types: Compose, Have, Solve, Cause, and Unknown. Their definitions and descriptions are provided in Table 2.

Table 1. Definitions of entity types

Entity Type	Symbol	Description
Fault system	FS	System-level components, such as the main transmission system, lubrication system, and tail transmission system.
Fault phenomenon	FP	Observable characteristics accompanying fault occurrence, including abnormal vibration and persistent noise.
Fault mode	FM	Degradation or failure modes of transmission components, such as long-term fatigue and seal failure.
Fault component	FC	Critical components of the transmission system, including gears, bearings, shafts, and couplings.
Repair measure	RM	Maintenance or remediation actions, such as process reinforcement or material replacement.

Table 2. Definitions of relation types

Relation Type	Symbol	Description
Compose	Compose	A fault system represents a higher-level concept composed of specific fault components; for example, a main gearbox system consists of clutches and gears.
Cause	Cause	A fault mode leads to specific system phenomena, such as insufficient lubrication causing overheating or continuous noise.
Have	Have	A specific component exhibits a particular fault phenomenon, such as persistent meshing noise in a gear or abnormal vibration in a structural part.
Solve	Solve	Maintenance actions applied to fault components, such as gear replacement or repair and oil filter cleaning.
Unknown	Unknown	The relationship between two entities is not yet explicitly defined and remains to be identified.

2.2 Annotation of Fault Text Data

Global pointer annotation is an efficient and flexible labeling strategy [18] that identifies the start and end boundaries of key textual spans within a sequence, thereby addressing nested and overlapping structures that are difficult to handle using conventional sequence labeling methods. As illustrated in Figure 2, for the sentence “The main reduction gear experiences tooth breakage”, a separate matrix is constructed for each entity type or relation type. Each matrix entry at position (i, j) represents the probability that the text span from position i to position j corresponds to a valid entity. This design allows entities such as “main reducer” and “main reducer gear” to be simultaneously assigned positive labels within the same structure, effectively resolving boundary conflicts associated with nested entities.

	主	减	齿	轮	出	现	断	齿
主	0	0	0	1	0	0	0	0
减		0	0	0	0	0	0	0
齿			0	0	0	0	0	0
轮				0	0	0	0	0
出					0	0	0	0
现						0	0	0
断							0	0
齿								0

(a) Head entity annotation

	主	减	齿	轮	出	现	断	齿
主	0	0	0	0	0	0	0	0
减		0	0	0	0	0	0	0
齿			0	0	0	0	0	0
轮				0	0	0	0	0
出					0	0	0	0
现						0	0	0
断							0	1
齿								0

(b) Tail entity annotation**Figure 2.** Illustration of global pointer annotation

Fault-related text segments from helicopter transmission systems were annotated using this scheme. The annotation results include sentence text, head entities, tail entities, and the relations linking them, as shown in Figure 3.

```

1 {
2   "id": "HTF_2024_Case_105",
3   "text": "尾减速器输入端的主动锥齿轮出现齿根严重疲劳裂纹，导致机身尾部出现高频异常振动及减速器滑油温度报警，需立即更换 该主动锥齿轮并对滑油系统进行磨粒分析。",
4   "entities": [
5     {"id": "1", "label": "FC", "start": 0, "end": 13, "text": "尾减速器输入端的主动锥齿轮"},
6     {"id": "2", "label": "FM", "start": 15, "end": 23, "text": "齿根严重疲劳裂纹"},
7     {"id": "3", "label": "PHN", "start": 26, "end": 39, "text": "机身尾部出现高频异常振动"},
8     {"id": "4", "label": "FP", "start": 40, "end": 49, "text": "减速器滑油温度报警"},
9     {"id": "5", "label": "RM", "start": 53, "end": 61, "text": "更换该主动锥齿轮"},
10    {"id": "6", "label": "RM", "start": 62, "end": 73, "text": "对滑油系统进行磨粒分析"}],
11   "relations": [
12     {"subject": "1", "predicate": "Compose", "object": "2"},
13     {"subject": "2", "predicate": "Cause", "object": "3"},
14     {"subject": "2", "predicate": "Cause", "object": "4"},
15     {"subject": "2", "predicate": "Solve", "object": "5"},
16     {"subject": "2", "predicate": "Solve", "object": "6"}]
17 }

```

Figure 3. Example of annotated JSON data

As illustrated in Figure 3, fault texts in helicopter transmission systems exhibit substantial structural complexity. The entity with $id = 1$, “driving bevel gear at the input end of the tail reducer”, spans 13 characters, and conventional word-based annotation methods are prone to fragmenting it into separate entities such as “tail reducer” and “bevel gear”, thereby losing essential semantic associations. The GP mechanism adopted in this study employs grid-based pointers to effectively cover such long-span entities. In addition, the entity with $id = 2$ simultaneously triggers two fault phenomena ($id = 3$ and $id = 4$) and corresponds to two repair measures ($id = 5$ and $id = 6$). This one-to-many correspondence requires the model not only to recognize entities accurately but also to integrate contextual information over long distances in order to capture complex dependency relationships.

3 RL-BGPNet Model Architecture

3.1 BERT-Based Preprocessing

Prior to model construction, the input text is preprocessed using the BERT model [19]. BERT is a pretrained language model based on the Transformer architecture, capable of exploiting bidirectional contextual information to learn rich linguistic knowledge and semantic representations. Its overall structure is illustrated in Figure 4.

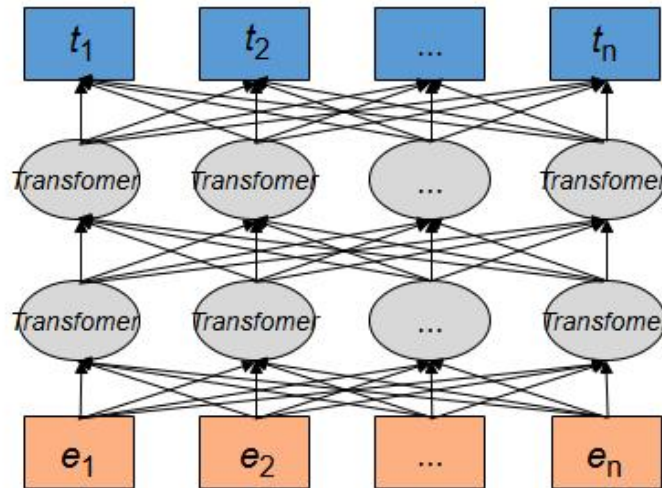


Figure 4. Architecture of the BERT preprocessing module

The original text is first tokenized and converted into subword units compatible with BERT. These tokens are then encoded through the BERT model to obtain context-dependent vector representations. By means of a multi-layer Transformer encoding structure with self-attention mechanisms, BERT jointly models both preceding and succeeding contextual information, producing embeddings that capture deep semantic dependencies. During encoding, each

subword token is mapped to a fixed-dimensional vector that integrates both its intrinsic semantic meaning and contextual cues, allowing for a more expressive representation of the input text. For example, in the sentence “The main reduction gearbox produces intermittent abnormal noise”, each subword unit (e.g., “main,” “reduction,” “gearbox,” “produces,” “intermittent,” “abnormal,” “noise”) is transformed into a vector, and these vectors are aggregated through self-attention to form contextualized representations.

$$X = \{x_1, x_2, \dots, x_n\} = \text{Tokenizer}(T) \quad (1)$$

where, T denotes the original input text, X represents the tokenized sequence, and n is the sequence length.

The token sequence X is subsequently fed into the BERT encoder, which outputs a hidden-state vector for each position. These hidden states encode contextual semantic information and reflect both token-level semantics and inter-token relationships:

$$H = \text{BERT}(X) = \{h_1, h_2, \dots, h_n\} \quad (2)$$

where, H denotes the BERT hidden-state matrix and d_h is the hidden dimension, typically set to 768.

3.2 Cross-Attention Mechanism with Entity Type Embeddings

Existing approaches often struggle to handle scenarios in which multiple overlapping relational triplets appear within a single sentence. As illustrated in Figure 5, the phrase “main reducer gear wear” forms two triplets, namely (“main reducer gear wear,” causes, “abnormal vibration”) and (“main reducer gear wear,” causes, “increased noise”), indicating that the same token simultaneously participates in different relational structures. Conventional encoding schemes typically generate a single contextual representation per token, making it difficult to distinguish its multiple semantic roles across different entities or relations, which leads to representational ambiguity.

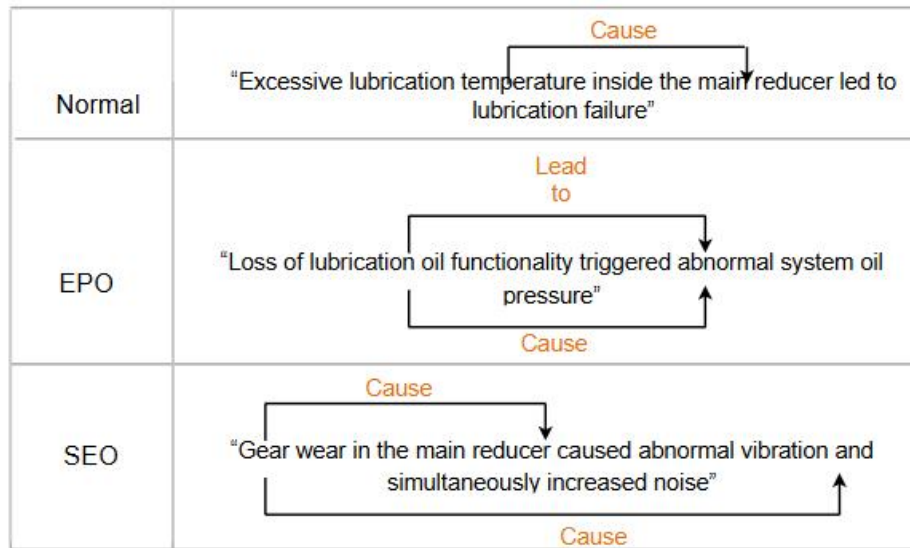


Figure 5. Illustration of entity overlap

Most joint extraction models primarily focus on identifying head and tail entities while underutilizing explicit entity type information. The pipeline-based PURE model demonstrates that incorporating entity types as explicit signals can substantially improve extraction performance [20], indicating that entity types can serve as effective prior knowledge for guiding feature learning and disambiguating overlapping entity boundaries. Motivated by this observation, a cross-attention mechanism with entity type embeddings is introduced [21], as shown in Figure 6. In this mechanism, entity type vectors are treated as queries, while the contextual semantic vectors produced by BERT serve as keys and values. Through cross-attention, textual features can be selectively indexed according to different entity types, enabling dynamic reconstruction of token representations that would otherwise be ambiguous. The resulting representations integrate contextual semantics with explicit type information, allowing more precise localization of head and tail entities under overlapping conditions and mitigating feature conflicts induced by entity overlap.

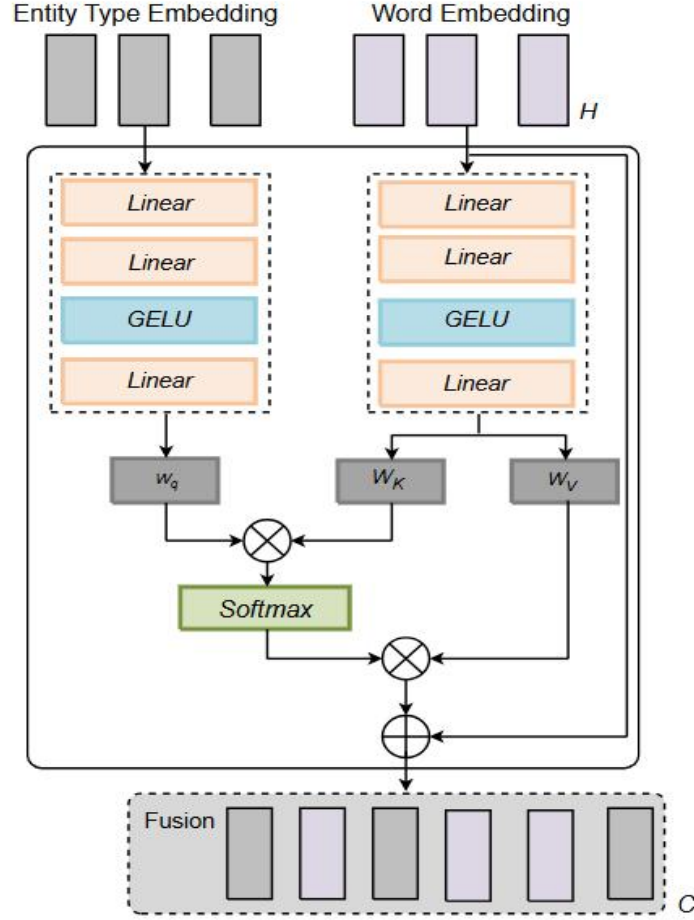


Figure 6. Cross-attention module with entity type embeddings

Let the predefined set of entity type labels be transformed into fixed-dimensional vectors through a BERT-based embedding layer (Entity Embedding). These entity type embeddings are then concatenated with the BERT hidden-state vectors:

$$E = \text{EntityEmbedding}(T_{\text{types}}) = \{e_1, e_2, \dots, e_m\} \quad (3)$$

where, E denotes the entity type embedding matrix, m is the number of entity types, and d_e is the embedding dimension.

For each contextual vector h_i output by BERT, attention scores with respect to all entity type embeddings are computed. Specifically, the projected query, key, and value vectors are defined as:

$$q_i = h_i W_Q, k_m = e_m W_K, v_m = e_m W_V \quad (4)$$

Attention score computation: the attention scores are obtained by computing the scaled dot-product similarity between q_i and k_m .

$$\text{score}_{i,m} = \frac{q_i^\top k_m}{\sqrt{d_k}} \quad (5)$$

where, d_k denotes the dimensionality of the key vectors to prevent excessively large dot products.

The attention weights are normalized using the Softmax function to yield the relevance of token i to each entity type m :

$$\alpha_{i,m} = \text{Softmax}_m(\text{score}_{i,m}) = \frac{\exp(\text{score}_{i,m})}{\sum_{k=1}^M \exp(\text{score}_{i,k})} \quad (6)$$

In these expressions, $\alpha_{i,m}$ quantifies the relevance strength between token i and entity type m .

Using the computed attention weights $\alpha_{i,m}$, a weighted summation is performed over the projected entity-type value vectors V_m , yielding a type-aware enhanced representation $\tilde{e}_i$ specific to the current token i .

$$c_i = \sum_{m=1}^M \alpha_{i,m} \mathbf{V}_m \quad (7)$$

This vector aggregates features from all entity types, modulated by their relevance to the current token. Finally, the original contextual vector h_i is fused with its corresponding type-aware enhancement vector to obtain the final type-aware contextual representation:

This representation aggregates features from all entity types and modulates them according to the attention intensity associated with token i .

Finally, the original BERT contextual vector h_i is fused with its corresponding type-aware enhanced vector $\tilde{e}_i$, resulting in the final type-aware contextual representation $\tilde{h}_i$.

$$\tilde{h}_i = \text{MLP}(h_i + c_i) \quad (8)$$

The resulting sequence $\tilde{H} = \{\tilde{h}_1, \dots, \tilde{h}_n\}$ is used as the input to the GPN.

3.3 Global Pointer Network (GPN) Decoding

The GPN addresses entity nesting by enumerating candidate spans in a matrix-based manner. As shown in Figure 7, in the sentence “Inspection revealed fatigue spalling of the main reducer gear, leading to excessive metal debris content in the lubricating oil”, the phrase “fatigue spalling” constitutes a sub-entity within the larger entity “main reducer gear fatigue spalling”. This forms a typical nested structure, where the former corresponds to a fault mode and the latter represents a complete fault event. In transmission system fault texts, entities frequently appear as composite phrases with hierarchical and overlapping characteristics. Sequence-labeling approaches, constrained by label transition dependencies, often fail to identify multiple granular entities simultaneously, leading to information loss or mislabeling. In contrast, the GPN constructs a start–end position matrix over all possible spans and independently evaluates each candidate, enabling unbiased, complete, and parallel recognition of nested entities.

Sentence 1: Fatigue spalling was identified on the main reducer gear, leading to an excessive concentration of metallic particles in the lubricating oil.

Sentence 2: Overheating and seizure of the tail drive shaft bearing were observed due to insufficient lubrication, necessitating immediate shutdown.

Sentence 3: A shear fracture was detected in the planetary carrier pin of the main reducer.

Figure 7. Illustration of nested entities

Within the reinforcement learning framework described in Section 4.1, the GPN functions as the policy network. Rather than producing deterministic labels, it outputs probability distribution matrices over all possible entity spans and relations, which together define the action space of the reinforcement learning agent. The decoding process is detailed as follows.

(1) Feature transformation with relative positional encoding

To accurately capture relative distance information between entity boundaries, rotary position embedding (RoPE) is incorporated prior to attention score computation. For each contextual vector $\tilde{h}_i$, linear projections are applied to generate query and key vectors, which are then combined with RoPE:

$$q_i = R_i(W_q \tilde{h}_i) \quad (9)$$

$$k_j = R_j \left(W_k \tilde{h}_j \right) \quad (10)$$

In these expressions, $W_q, W_k \in \mathbb{R}^{d \times d}$ denotes learnable projection matrices, while R_i and R_j are the rotation matrices applied at positions i and j , respectively. Owing to the inner-product properties of the rotary transformation, the interaction between the transformed vectors $q_i^\top k_j$ naturally encodes relative positional information $(j - i)$.

(2) Entity recognition policy generation

Entity recognition is formulated as a fully connected span prediction problem. The policy network aims to generate a probability matrix $L \times L$, where each entry (i, j) denotes the probability that the span from token i to token j constitutes a valid entity.

The entity scoring function S_{entity} is defined using a scaled dot product:

$$s_{\text{entity}}(i, j) = \frac{\tilde{q}_i^\top \tilde{k}_j}{\sqrt{d}} \quad (11)$$

To accommodate multi-type entity recognition, the parameterization is extended for each entity type $m (m \in \{1, \dots, M\})$. Within the reinforcement learning framework, the resulting scores are converted into action selection probabilities. Specifically, the Sigmoid function is applied to map the scores into the interval $[0, 1]$, yielding the corresponding policy distribution π_θ^{ent} .

$$P(y_{i,j}^m = 1 | s) = \sigma(s_{\text{entity}}^m(i, j)) = \frac{1}{1 + e^{-s_{\text{entity}}^m(i, j)}} \quad (12)$$

where, $\sigma(\cdot)$ denotes the Sigmoid function. If $P(y_{i,j}^m = 1 | s) > \delta$ exceeds a predefined threshold, the agent treats the span (i, j) as an entity of type m . This probability distribution is subsequently used for action sampling during reinforcement learning.

(3) Relation classification policy generation

After entity candidates are obtained, the policy network further determines the semantic relations between pairs of entities, which is likewise implemented through the global pointer mechanism. When assessing whether a relation exists between entity E_{sub} (subject) and entity E_{obj} (object), the head representation of the subject $h_{\text{sub}}^{\text{head}}$ and the head representation of the object $h_{\text{obj}}^{\text{head}}$ are taken as the input features.

For a specific relation type r , the directional score from subject to object is computed as:

$$s_{\text{relation}}^r(\text{sub}, \text{obj}) = (W_{q,r}^{\text{head}})^\top R_{\text{obj-sub}} W_{k,r}^{\text{head}} \quad (13)$$

In these expressions, $R_{\text{obj-sub}}$ denotes the interaction matrix between the subject and object entities for relation. Similarly, after Sigmoid activation, the resulting score forms the policy distribution π_θ^{rel} for relation extraction.

$$P(r | E_{\text{sub}}, E_{\text{obj}}) = \sigma(s_{\text{relation}}^r(\text{sub}, \text{obj})) \quad (14)$$

Through these transformations, the complete policy $\pi_\theta(a | S)$ under state S can be decomposed into the joint distribution of entity recognition and relation classification policies. Each forward pass of the model effectively produces a high-dimensional action probability map. During subsequent reinforcement learning, actions are sampled from these distributions to generate concrete entity–relation triplets, which interact with the environment to obtain corresponding rewards.

4 Decoding Optimization and Training Strategy

Although the GPN resolves entity nesting and overlap through matrix-based enumeration, conventional cross-entropy loss during training emphasizes local token-pair matching and tends to produce partially correct yet noisy triplets. In contrast, reinforcement learning employs the F1 score—a task-level global metric—as the reward signal, guiding the model to prioritize triplets that are semantically consistent at the sequence level. Moreover, reinforcement learning is well suited to engineering texts with high triplet density; policy-gradient optimization improves discrimination in one-to-many relations and reduces decision bias introduced by overlapping entities.

4.1 Reinforcement Learning-Based Decoding Optimization

In practical inference and training, decoding with the GPN ultimately requires transforming continuous probabilities into discrete action sequences via sampling or truncation. Accordingly, an reinforcement learning-based optimization framework is constructed on top of the global pointer policy network, with the F1 score used directly as the optimization objective, as illustrated in Figure 8. In this framework, discrete extraction actions are treated as agent decisions, and policy-gradient methods are applied to maximize the expected F1 score, thereby establishing an end-to-end pathway from probabilistic prediction to metric-oriented optimization.

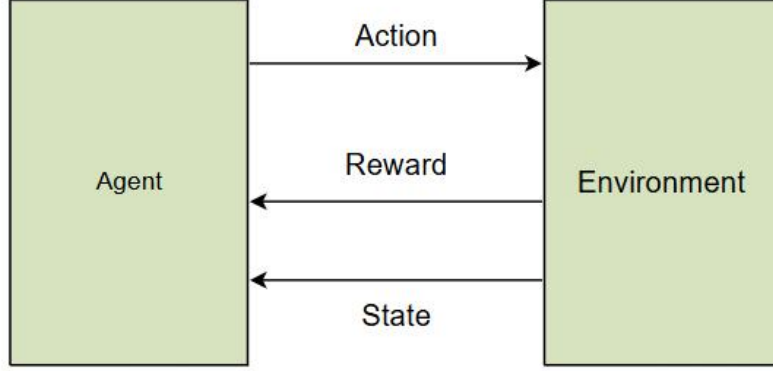


Figure 8. Schematic of the reinforcement learning framework

Within the framework shown in Figure 8, the global pointer decoding model described in Section 3.3 is regarded as the agent, while the training dataset and its corresponding ground-truth annotations constitute the interactive environment. Contextual features extracted by the encoder are treated as the state, whereas the binary mask matrices sampled from the policy distribution and the decoded entity–relation triplets are treated as actions. Policy-gradient optimization is then applied to directly maximize the expected return using the F1 score as the reward signal.

Given the parallel-output characteristic of the GPN, the entity–relation joint extraction task is formulated as a reinforcement learning policy optimization process. The state–action–reward triplet (S, A, R) is defined as follows.

(1) State S

The state consists of the current input sentence and its contextual semantic representation. For a given input text, the state s corresponds to the contextual matrix $H \in \mathbb{R}^{I \times d}$ output by the encoder.

(2) Action A

An action a is defined as a set of entity spans and relation triplets predicted by the policy network under the current state. Owing to the parallel output of the GPN, which produces a probability matrix over all candidate spans and relations, the action a is concretely instantiated as a binary mask matrix obtained by Bernoulli sampling from the policy distribution π_θ .

$$a = \{(e_{\text{head}}, e_{\text{tail}}, r) \mid e_{\text{head}}, e_{\text{tail}} \in \text{Entity}, r \in R\} \quad (15)$$

(3) Policy π_θ

From the reinforcement learning perspective, the policy network corresponds to the parameterized GPN defined in Section 3.3, with parameters θ . For a given state s , the policy function $\pi_\theta(a \mid s)$ defines the probability density of generating action a :

$$\pi_\theta(a \mid s) = P_{\text{GPN}}(a \mid H; \theta) \quad (16)$$

(4) Reward R

The reward function serves as the core optimization signal in reinforcement learning. To directly optimize extraction performance, the reward is defined as the F1 score between the predicted triplet set a and the ground-truth annotation set G :

$$R(a, G) = \text{F1}(a, G) = \frac{2 \cdot P \cdot R}{P + R} \quad (17)$$

where, precision P and recall R are computed by comparing the predicted triplet set a with the reference set G .

4.2 Objective Function and Gradient Design

Given the parameter set θ of the policy π_θ , the objective function is constructed to identify parameters θ^* that maximize the expected cumulative reward. The optimization objective $J(\theta)$ is defined as:

$$J(\theta) = \mathbb{E}_{a \sim \pi_\theta(-|s)} [R(a, G)] \quad (18)$$

which, $\mathbb{E}_{a \sim \pi_\theta(+|s)}$ represents the maximization of the expected F1 score of actions sampled from the current policy distribution.

Because the reward function R (i.e., the F1 score) is non-differentiable with respect to the model parameters θ , direct backpropagation via the chain rule is not applicable. Therefore, the REINFORCE algorithm [22] is adopted to estimate the policy gradient. According to the policy gradient theorem, the gradient of the objective function $J(\theta)$ with respect to θ can be expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{a \sim \pi_\theta(+|s)} [R(a, G) \cdot \nabla_\theta \log \pi_\theta(a | s)] \quad (19)$$

In practice, this expectation is approximated by sampling K actions using Bernoulli sampling:

$$\nabla_\theta J(\theta) \approx \frac{1}{K} \sum_{k=1}^K R(a_k, G) \cdot \nabla_\theta \log \pi_\theta(a_k | s) \quad (20)$$

In this expression, $\nabla_\theta \log \pi_\theta(a_k | s)$ indicates the direction of parameter updates, guiding the optimization to increase the probability of generating a_k . The scalar reward term serves as the update weight $R(a_k, G)$: if a_k yields a high F1 score, its probability is substantially reinforced; conversely, actions associated with low F1 scores are suppressed during parameter updates.

$$\nabla_\theta J(\theta) \approx \frac{1}{K} \sum_{k=1}^K (R(a_k, G) - b) \cdot \nabla_\theta \log \pi_\theta(a_k | s) \quad (21)$$

In this model, the baseline b is set as the moving average of rewards over all sampled actions within the current training batch. When $R(a_k) > b$, the gradient term $(R - b)$ is positive and reinforces the corresponding action; otherwise, the gradient penalizes the action.

4.3 Loss Function and Training Strategy

The total loss of the model consists of a supervised learning loss L_{SL} and a reinforcement learning loss L_{RL} .

(1) Supervised learning loss

During the supervised phase, the objective is to maximize the likelihood of the ground-truth annotations. The probability matrices produced by the GPN correspond to a multi-label classification problem. A variant of binary cross-entropy loss is adopted as the supervised loss:

$$L_{SL}(\theta) = - \sum_{(i,j) \in \Omega} [y_{i,j} \log P_\theta(i, j) + (1 - y_{i,j}) \log (1 - P_\theta(i, j))] \quad (22)$$

where, Ω denotes the set of all possible entity or relation spans, $y_{i,j} \in \{0, 1\}$ represents the ground-truth label, and $P_\theta(i, j)$ denotes the predicted probability output by the GPN. To address severe class imbalance, a sparse version of the cross-entropy loss is employed.

(2) Reinforcement learning loss

As described in Section 4.2, reinforcement learning aims to maximize the expected reward $J(\theta)$. Within a gradient-based optimization framework, this objective is implemented by minimizing its negative form:

$$L_{RL}(\theta) = -J(\theta) \approx - \sum_{k=1}^K (R(a_k, G) - b) \log \pi_\theta(a_k | s) \quad (23)$$

To ensure training stability and efficient convergence, the overall training procedure is summarized in Figure 9.

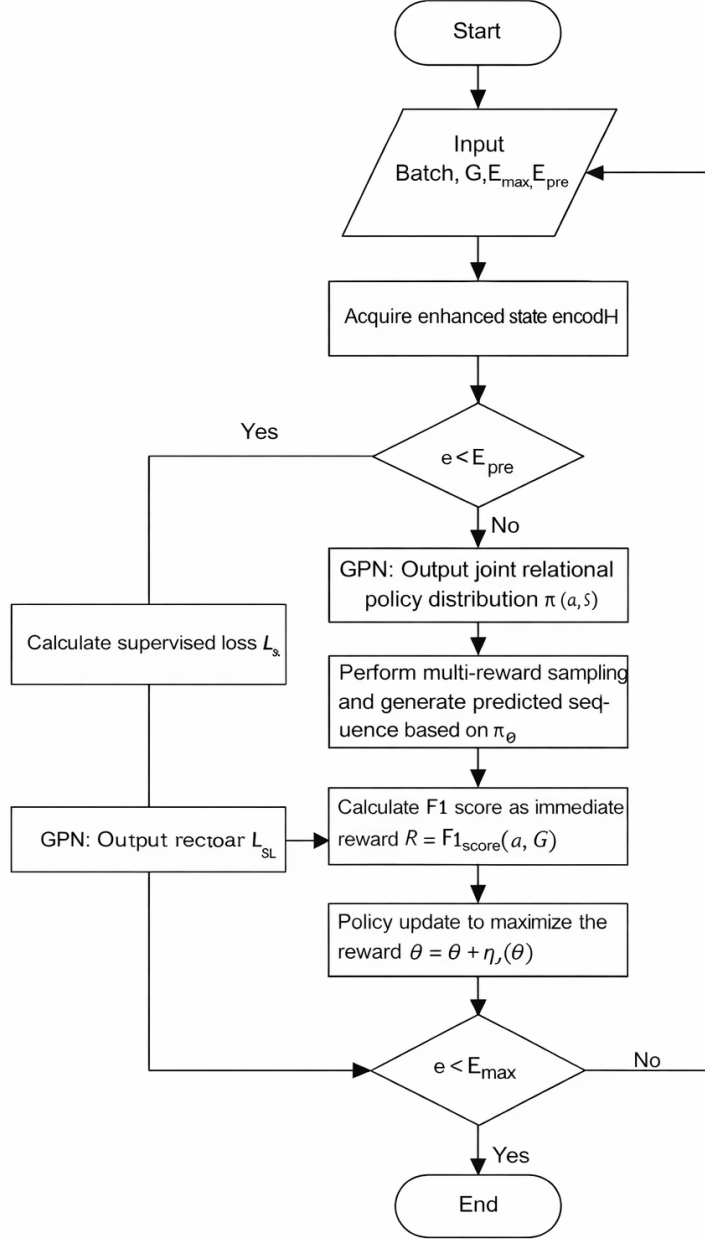


Figure 9. Training workflow

Step 1: Input the encoded states, training batches, and ground-truth labels, and obtain enhanced semantic state representations H via the BERT encoder.

Step 2: Determine whether the current epoch index e is less than or equal to the warm-up threshold E_{pre} . If $e \leq E_{pre}$, proceed to Step 3 to facilitate rapid convergence to a reasonable parameter region and mitigate cold-start issues in early reinforcement learning training; otherwise, proceed to Step 4 to directly optimize the non-differentiable evaluation metric.

Step 3: Compute the supervised loss L_{SL} based on state H and update parameters θ via gradient descent to complete foundational feature learning.

Step 4: Generate the joint policy distribution $\pi(a/s)$ over entities and relations using the GPN, and perform Bernoulli sampling to obtain discrete prediction action sequences.

Step 5: Compare the sampled actions with the ground-truth labels G , compute the F1 score as the immediate reward R , and update parameters using policy-gradient optimization to maximize the expected return $J(\theta)$.

5 Entity-Relation Joint Extraction Based on RL-BGPNNet

The overall architecture of the proposed entity-relation joint extraction model integrating reinforcement learning and a GPN (RL-BGPNNet) is illustrated in Figure 10. Within this framework, the BERT encoder, the cross-attention module, and the GPN collectively constitute the agent, while the ground-truth annotations and evaluation metrics form the interactive environment. The operational logic of the model follows a closed-loop paradigm of perception-decision-feedback.

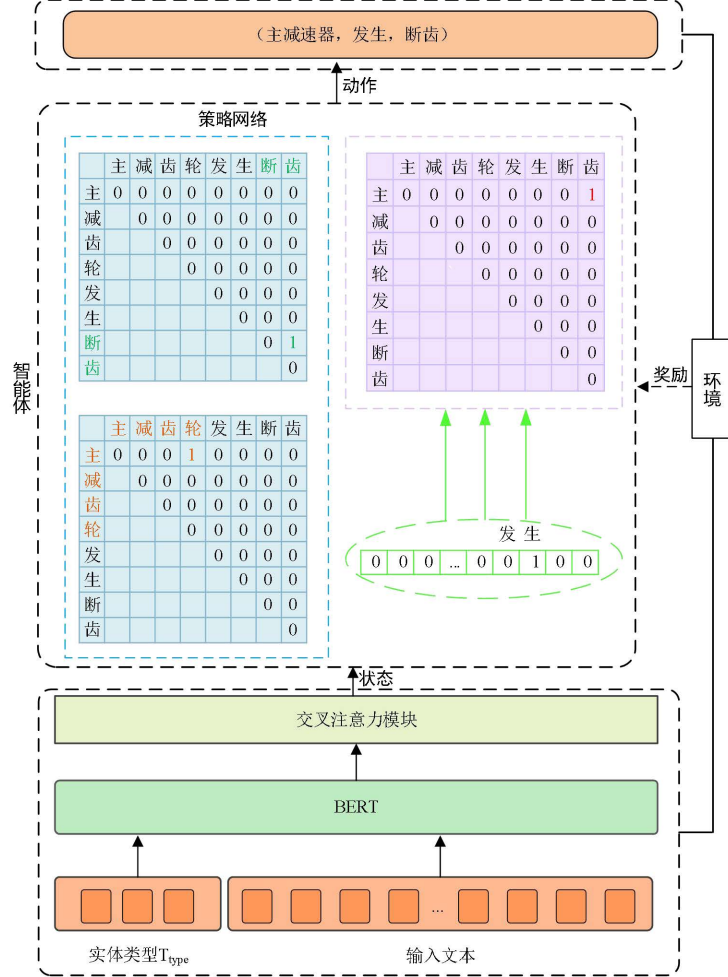


Figure 10. Architecture of the RL-BGPNNet-based entity-relation joint extraction model

The agent processes the input text and predefined entity types at the token level. Through the BERT preprocessing module, the input is transformed into low-dimensional semantic feature vectors. To fully exploit entity type information, a cross-attention mechanism is applied to integrate type-aware features and generate enhanced state representations. Subsequently, the GPN is employed as the policy network, producing probability distribution matrices over entity start-end positions and relation connections conditioned on the current state. Discrete binary mask matrices are then generated as actions and decoded into predicted triplets. The predicted results are submitted to the environment, where they are compared with ground-truth annotations to compute the F1 score as the reward. Gradients are propagated back to the agent via a policy-gradient algorithm, enabling direct optimization of non-differentiable evaluation metrics. The overall extraction procedure is depicted in Figure 11.

The specific implementation steps are summarized as follows.

Step 1: Structured and unstructured data are collected, and entity and relation types are defined according to the semantic characteristics of the helicopter transmission system fault domain.

Step 2: After data cleaning and filtering, a joint extraction dataset is constructed using the global pointer annotation scheme. The dataset is then partitioned into training and testing subsets.

Step 3: The joint extraction training set and the predefined entity types are jointly fed into the RL-BGPNNet model. Following BERT-based preprocessing, the representations enter the entity-type cross-attention module for deep interaction and fusion. The GPN then performs parallel computation of scoring matrices for head entity recognition

and relation–tail entity joint identification. During decoding, the REINFORCE algorithm is incorporated to jointly optimize entity recognition and relation prediction under the reinforcement learning framework, yielding predicted triplets.

Step 4: The joint extraction test set is input into the trained RL-BGPNet model to evaluate extraction accuracy and robustness.

Step 5: Entity–relation–entity triplets are generated, enabling the preliminary construction of the knowledge extraction graph.

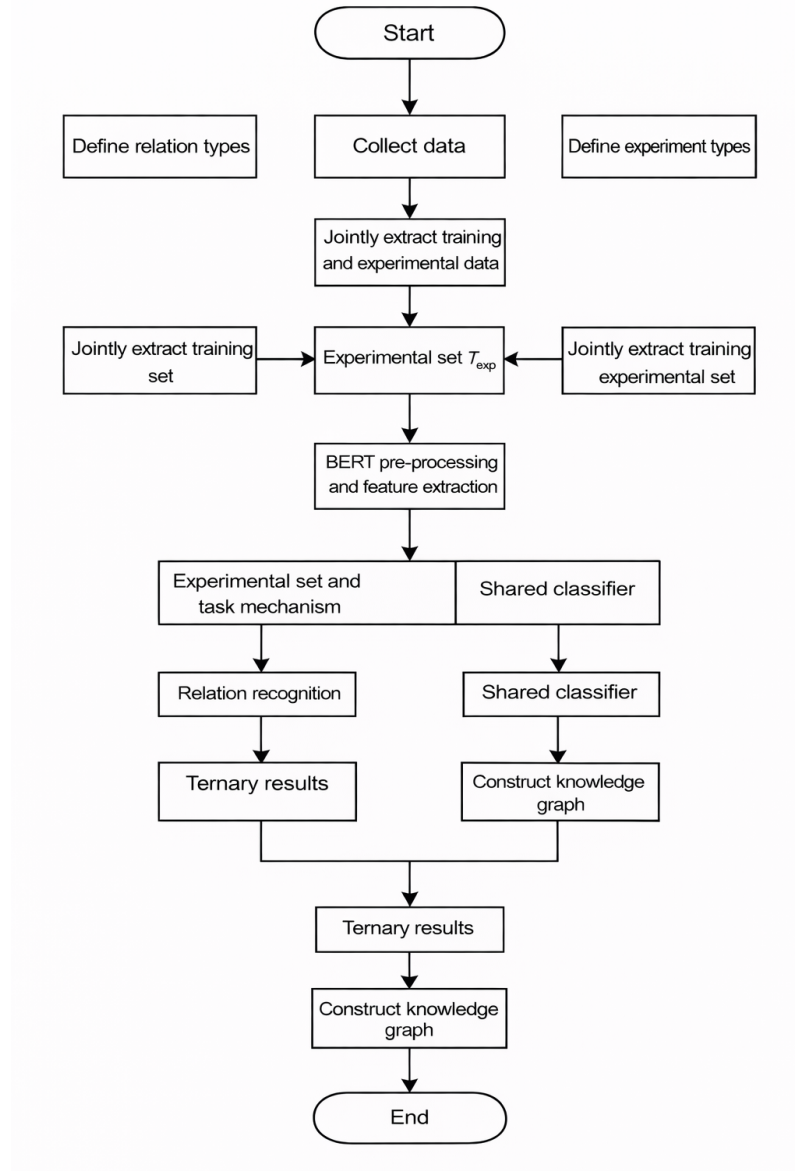


Figure 11. Workflow of entity–relation joint extraction based on RL-BGPNet

6 Experimental Validation and Analysis

6.1 Evaluation on Public Datasets

6.1.1 Experimental data

To evaluate the effectiveness and objectivity of the proposed RL-BGPNet model, comparative experiments were conducted on two public benchmark datasets containing a substantial number of overlapping entities: CMeIE [22] and NYT [23]. Detailed statistics on entity overlap in these datasets are summarized in Table 3.

Table 3 presents detailed information on the selected datasets. CMeIE, Chinese Medical Entity and Relation Extraction, is a widely used Chinese benchmark dataset for joint entity–relation extraction in the medical domain. It contains approximately 22,000 text instances, with 17,641 sentences in the training set and 4,683 sentences in the

test set, among which 2,158 and 5,236 sentences correspond to EPO and SEO overlapping patterns, respectively. The NYT dataset is derived from the New York Times corpus and was originally constructed for natural language generation tasks. In this study, a reprocessed version, NYT, is employed, which includes 30,872 and 10,096 sentences with EPO and SEO overlapping patterns, respectively. Each instance in NYT is associated with one or more standard reference triplets.

Table 3. Statistics of the CMeIE and NYT datasets

Data type	CMeIE		NYT	
	Training Set	Test Set	Training Set	Test Set
Normal	10247	2186	39597	2568
EPO	2158	1024	30872	1380
SEO	5236	1473	10096	1052
Total	17641	4683	56195	5000

6.1.2 Experimental settings and evaluation metrics

For consistency in experimental comparison, the learning rate of BERT was set to 2×10^{-5} , while the learning rate of the GPN was set to 2×10^{-4} . The dropout rate was fixed at 0.1, the number of training epochs was set to 100, the hidden dimension hyperparameter d was set to 256, and the batch size was set to 16. The warm-up parameter E_{pre} was set to 5, the sampling number k was set to 64, and the weighting coefficient of the joint loss function λ was set to 0.2.

Precision (Precision), recall (Recall), and the F1 score (F1) were adopted as the primary evaluation metrics. A prediction was considered correct only when the head entity boundary and type, the tail entity boundary and type, and the relation type in a triplet (Subject, Relation, Object) were all correctly identified. The corresponding computation formulas are given in Eqs. (24), (25), and (26), respectively:

$$\text{Precision} = \frac{\text{Number of correctly predicted triplets}}{\text{Number of triplets predicted by the model}} \times 100\% \quad (24)$$

$$\text{Recall} = \frac{\text{Number of correctly predicted triplets}}{\text{Number of ground-truth triplets}} \times 100\% \quad (25)$$

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (26)$$

The F1 score provides a balanced assessment by jointly considering precision and recall; values closer to 1 indicate better overall performance.

6.1.3 Baseline methods

To assess the performance of the proposed entity–relation joint extraction approach, four strong baseline models from recent years were selected for comparison:

1. CoPyRE [24]: This model adopts a Seq2Seq encoder–decoder framework and introduces a copy mechanism to address triplet overlap. The extraction process is formulated as a generation task, where relation categories are generated first, followed by copying the corresponding entities from the original text to produce complete triplets.

2. CasRel: This method constructs a cascade pointer network architecture that models relations as functional mappings from head entities to tail entities. The model first identifies the set of head entities and subsequently determines the corresponding relations and tail entities.

3. TPLinker: This approach proposes a single-stage extraction paradigm based on token-pair linking. Through a handshaking tagging mechanism, entity recognition and relation extraction are unified as link prediction in a matrix representation. It effectively handles nested entities and overlapping relations while fundamentally avoiding the exposure bias commonly observed in cascade models.

4. GPLinker: As one of the current state-of-the-art models, GPLinker leverages global pointer techniques to substantially improve entity boundary detection and relation classification accuracy. Owing to its strong feature extraction capability and generalization performance, it is selected as the baseline architecture in this study, upon which targeted enhancements and optimizations are introduced.

6.1.4 Experimental results and analysis

(1) Comparative Experiments

The experimental results of RL-BGPNet and the baseline models on the CMeIE and NYT datasets are reported in Table 4 and Table 5, respectively.

As shown in Table 4 and Table 5, RL-BGPNet achieves precision, recall, and F1 scores of 91.17%, 90.26%, and 90.71% on the NYT dataset, respectively. Compared with the four baseline models, RL-BGPNet consistently yields higher precision and F1 scores. Relative to the strongest baseline, GPLinker, precision and F1 are improved by 4.11% and 3.65% on the CMeIE dataset, and by 1.94% and 3.09% on the NYT dataset, respectively. These gains can be attributed to the integration of the entity–relation attention mechanism, which strengthens the interaction between entity type information and contextual text representations, as well as to the reformulation of decoding as a reward-driven decision process. By optimizing entity integrity and relation correctness through an F1-based reward, the model is guided toward predictions that are more consistent with global task objectives. The results demonstrate the effectiveness of RL-BGPNet.

Table 4. Experimental results on the CMeIE dataset

Model	Precision (%)	Recall (%)	F1 (%)
CopyRE	59.14	56.53	57.80
CasRel	64.31	55.86	59.79
TPLinker	65.89	58.45	61.95
GPLinker	68.21	58.19	62.80
RL-BGPNet (proposed)	72.32	61.46	66.45

Table 5. Experimental results on the NYT dataset

Model	Precision (%)	Recall (%)	F1 (%)
CopyRE	61.12	57.61	59.31
CasRel	81.54	75.76	78.55
TPLinker	87.02	83.17	85.05
GPLinker	89.23	86.04	87.62
RL-BGPNet (proposed)	91.17	90.26	90.71

(2) Ablation Experiments

To further examine the contribution of individual components, ablation studies were conducted on the CMeIE and NYT datasets. Three key components were removed in turn: the BERT preprocessing module, the relation embedding attention (REA) module, and the reinforcement learning optimization strategy. The results are presented in Table 6.

Table 6. Ablation study results of RL-BGPNet

Dataset	Model	Precision (%)	Recall (%)	F1 (%)
CMeIE	RL-BGPNet	72.32	55.46	62.78
	BERT	58.21	41.35	48.35
	REA	68.94	52.17	59.39
	Reinforcement learning	60.18	57.82	58.98
NYT	RL-BGPNet	91.17	90.26	90.71
	BERT	78.45	76.31	77.37
	REA	89.23	88.97	89.10
	Reinforcement learning	83.21	91.54	87.18

Removing the BERT preprocessing module leads to substantial performance degradation, with F1 decreasing by 14.43% on CMeIE and 13.34% on NYT. This indicates that contextualized representations produced by BERT are essential for accurately identifying entity boundaries and semantic relations, particularly in texts containing complex terminology and sentence structures where static embeddings fail to capture long-range dependencies.

The removal of the entity-type cross-attention module also negatively affects performance, with F1 decreasing by 5.11% on CMeIE and 1.26% on NYT. These results suggest that REA effectively enhances the model’s ability to capture and distinguish complex entity–relation interactions.

When the reinforcement learning optimization strategy is removed, precision on CMeIE decreases from 72.32% to 60.18%, while recall slightly increases to 57.82%, resulting in an overall F1 reduction of 5.78%. A similar trend is observed on NYT, where precision decreases from 91.17% to 83.21% and F1 drops by 3.53%. These findings indicate that the primary role of reinforcement learning lies in suppressing erroneous triplets and enforcing global

optimization. Conventional cross-entropy loss emphasizes local label matching and tends to generate partially correct but noisy triplets, whereas F1-driven policy optimization encourages global consistency and substantially improves prediction precision.

(3) Overlap Analysis

Entity overlap represents a major challenge in joint entity–relation extraction, typically manifested as shared characters or nested spans that complicate boundary identification. Such coupling of semantic and positional information often leads to misclassification or omission, thereby degrading precision, recall, and F1.

To further assess the effectiveness of RL-BGPNet under complex overlap conditions, performance was analyzed across three overlap categories on both CMeIE and NYT. The corresponding F1 scores are reported in Table 7.

Table 7. F1 scores under different overlap patterns (%)

Model	CMeIE			NYT		
	Normal	SEO	EPO	Normal	SEO	EPO
CoPyRE	58.23	35.47	28.64	61.59	55.26	52.18
CasRel	65.31	68.45	45.28	85.12	87.29	65.53
TPLinker	66.56	70.14	68.57	88.21	90.13	89.58
GPLinker	67.19	71.22	69.86	88.54	90.87	90.25
RL-BGPNet (proposed)	68.25	75.48	76.15	89.16	92.35	92.83

Note: SEO = Single Entity Overlap; EPO = Entity Pair Overlap.

As the overlap pattern shifts from Normal to Entity Pair Overlap (EPO) and complexity increases, the performance of CoPyRE, CasRel, and TPLinker deteriorates markedly. In contrast, RL-BGPNet exhibits strong robustness. Under the Normal setting, RL-BGPNet performs comparably to GPLinker, indicating that most models approach performance saturation on simple sentences. However, under Single Entity Overlap (SEO) and EPO conditions, the advantage of RL-BGPNet becomes pronounced. In particular, for EPO on CMeIE, RL-BGPNet achieves an F1 score of 76.15%, exceeding GPLinker by 6%. This improvement demonstrates that the reinforcement learning optimization effectively captures fine-grained dependencies among entities, enabling more accurate discrimination when multiple relation labels coexist for the same entity pair.

(4) Impact of Triplet Density

In practical applications, text information density varies widely, and individual sentences may contain multiple interrelated triplets. To evaluate robustness under high-density conditions, the test sets were partitioned into five groups according to the number of triplets per sentence ($N = 1$ to $N \geq 5$). The results are shown in Table 8 and Table 9.

Table 8. F1 performance on CMeIE under different triplet counts (%)

Model	N = 1	N = 2	N = 3	N = 4	N ≥ 5
CoPyRE	60.57	54.23	45.38	38.14	25.69
CasRel	66.82	64.56	60.21	55.48	42.35
TPLinker	68.29	67.54	65.17	62.83	54.52
GPLinker	69.16	68.41	66.25	63.59	56.13
RL-BGPNet (proposed)	70.25	70.18	69.56	67.87	63.44

Table 9. F1 performance on NYT under different triplet counts (%)

Model	N = 1	N = 2	N = 3	N = 4	N ≥ 5
CoPyRE	85.23	78.47	65.52	55.29	40.14
CasRel	88.58	87.21	85.66	80.45	72.53
TPLinker	89.84	89.28	88.59	86.42	82.16
GPLinker	90.17	89.53	88.81	87.18	83.24
RL-BGPNet (proposed)	90.86	90.65	90.27	89.59	88.48

As the number of triplets increases, extraction performance declines for all models due to the growing complexity of semantic structures and the increased likelihood of entity overlap and interference. Despite this inevitable degradation, RL-BGPNet demonstrates the highest stability, particularly for $N \geq 3$. On CMeIE, RL-BGPNet exceeds GPLinker by 1.12% in F1 when $N = 1$, and maintains an F1 score of 63.44% when $N \geq 5$, outperforming the second-best model by 7.13%. In contrast, CasRel and CoPyRE fall below 43%, and GPLinker declines to 56.13%. These results indicate

that RL-BGPNet effectively resolves dense semantic dependencies and mitigates missed extractions in high-density scenarios.

On the NYT dataset, RL-BGPNet similarly exhibits strong long-tail robustness. Even in the extreme case of $N \geq 5$, the F1 score remains at 88.48%, demonstrating that the model maintains global contextual awareness and avoids feature confusion as the number of extraction targets increases.

6.2 Entity-Relation Extraction Experiments on the Helicopter Transmission System Fault Dataset

To evaluate the effectiveness of the proposed RL-BGPNet model in the domain of helicopter transmission system fault analysis, the dataset constructed in Section 2 was randomly split into training, validation, and test sets at a ratio of 8:1:1. The detailed statistics are summarized in Table 10.

Table 10. Statistics of the helicopter transmission system fault dataset

Dataset	Type	Training Set	Validation Set	Test Set
Helicopter transmission system fault dataset	Number of sentences	1,400	175	175
	Number of entities	4,500	563	563
	Number of relation triplets	7,620	953	953
	Average sentence length	66.2	65.3	65.1

As shown in Table 10, the dataset contains approximately 3.4 entities and 5.4 relation triplets per sentence on average, indicating a relatively high information density. This setting is suitable for assessing the applicability of the proposed model under conditions involving complex entity overlap and varying numbers of triplets.

6.2.1 Experimental settings and evaluation metrics

To avoid truncation of long-range dependencies, the maximum input sequence length was set to 70. After configuring the remaining key parameters of RL-BGPNet for the helicopter transmission system fault domain, the final hyperparameter settings are listed in Table 11.

Table 11. Hyperparameter settings

Parameter	Value
Epoch	100
Batch size	8
Dropout rate	0.5
Learning rate warm-up ratio	0.1
Number of attention heads	12
RoPE size	64
Learning rate	2e-4
Reward	F1-Score
Discount factor γ	0.93
Reinforcement learning loss weight λ	0.2

6.2.2 Experimental results and analysis

(1) Comparative Experiments

The comparison results between RL-BGPNet and the baseline models on the helicopter transmission system fault dataset are presented in Table 12.

As indicated in Table 12, RL-BGPNet achieves a precision of 92.45%, a recall of 91.30%, and an F1 score of 91.87% on the helicopter transmission system fault dataset, outperforming all four baseline models. Compared with the second-best model, improvements of 2.77%, 3.15%, and 2.96% are observed in precision, recall, and F1, respectively. These gains can be attributed to the entity-type embedding attention mechanism, which enables more accurate identification of domain-specific entities, as well as to the reinforcement learning optimization framework, which provides positive guidance during decoding and leads to consistent performance improvements across metrics.

Overall, relative to existing GPN-based state-of-the-art models, the proposed approach introduces entity-type cross-attention to alleviate feature conflicts caused by entity overlap and reinforcement learning to achieve global performance optimization, whereas GPLinker relies solely on local supervision signals. Compared with the single-stage TPLinker model, the use of a GPN in place of a token-pair matrix reduces computational complexity from

$O(N^2)$ to a more manageable parallel formulation, while the reinforcement learning module mitigates sparsity issues in long-document scenarios. In contrast to cascade-based models such as CasRel, the proposed approach avoids exposure bias arising from the use of gold subjects during training and predicted subjects during inference. Through single-stage decoding combined with global reinforcement learning optimization, extraction stability is improved for complex engineering texts. These results indicate that the proposed model maintains strong performance when confronted with entity–relation overlap and complex long-range semantic dependencies in helicopter transmission system fault data, and is suitable for knowledge extraction and KG construction tasks.

Table 12. Comparison of entity–relation extraction performance on the helicopter transmission system fault dataset

Model	Precision (%)	Recall (%)	F1 (%)
CopyRE	79.25	76.43	77.82
CasRel	85.37	83.14	84.24
TPLinker	88.52	87.26	87.89
GPLinker	89.68	88.15	88.91
RL-BGPNet (Ours)	92.45	91.30	91.87

(2) Model Performance Analysis

From an engineering deployment perspective, additional experiments were conducted on the helicopter transmission system fault dataset to analyze performance differences among models. The computational complexity of RL-BGPNet mainly arises from three components:

BERT encoder: time complexity $O(N \times d_h^2)$, where N is the sequence length and $d_h = 768$, consistent with baseline models;

Entity-type cross-attention: time complexity $O(N \times m \times d_e)$, where $m=5$ entity types and $d_e = 256$. Given the small value of m , the additional overhead is negligible;

GPN with reinforcement learning: the original GPN has a time complexity of $O(N^2 \times k)$, where $k = 5$ relation types. Through RoPE positional encoding and Bernoulli sampling ($K = 64$), this is reduced to $O(N \times k)$, while the policy-gradient computation in reinforcement learning incurs an additional cost of $O(K \times N)$.

The overall time complexity is therefore $O(N \times d_h^2 + N \times k)$. Compared with TPLinker ($O(N^2)$), an efficiency improvement of approximately 30% is achieved in long-text scenarios ($N = 70$). In terms of space complexity, sparse matrix storage in the GPN reduces GPU memory usage by approximately 40% relative to TPLinker. A detailed comparison of training and inference resource consumption under identical hardware conditions is provided in Table 13.

Table 13. Model performance comparison

Model	Training Time Per Epoch (s)	Inference Time Per Sentence (ms)	GPU Memory Usage (GB)
CoPyRE	12.8	8.5	14.2
CasRel	10.3	6.2	11.5
TPLinker	15.7	9.8	18.7
GPLinker	9.6	5.7	10.8
RL-BGPNet (proposed)	11.2	7.3	12.4

As shown in Table 13, although RL-BGPNet incorporates a reinforcement learning module, the efficiency gains achieved by the GPN limit the increase in training and inference time to 16.7% and 28.1%, respectively, relative to GPLinker. GPU memory usage increases by 14.8%, remaining within an acceptable range for practical engineering applications.

6.3 Knowledge Graph Construction

Based on the extraction of “entity–relation–entity” triplets from helicopter transmission system fault data using the RL-BGPNet model, a domain-specific KG is constructed by importing all validated triplets into the Neo4j graph database via the Cypher query language, as illustrated in Figure 12.

By extracting triplets such as fault component–fault mode–repair measure from fault-related texts, a structured fault KG is established. This graph can be directly integrated into intelligent helicopter maintenance systems to support rapid graph-based matching of inference paths from fault phenomena $\rightarrow$ fault causes $\rightarrow$ repair solutions, thereby assisting fault diagnosis and decision-making. In addition, the KG facilitates the automated consolidation of dispersed domain knowledge from maintenance manuals and patent documents, enabling the formation of reusable

fault-handling rule repositories. Such integration reduces the cost of knowledge acquisition for frontline maintenance personnel and supports systematic engineering knowledge management.

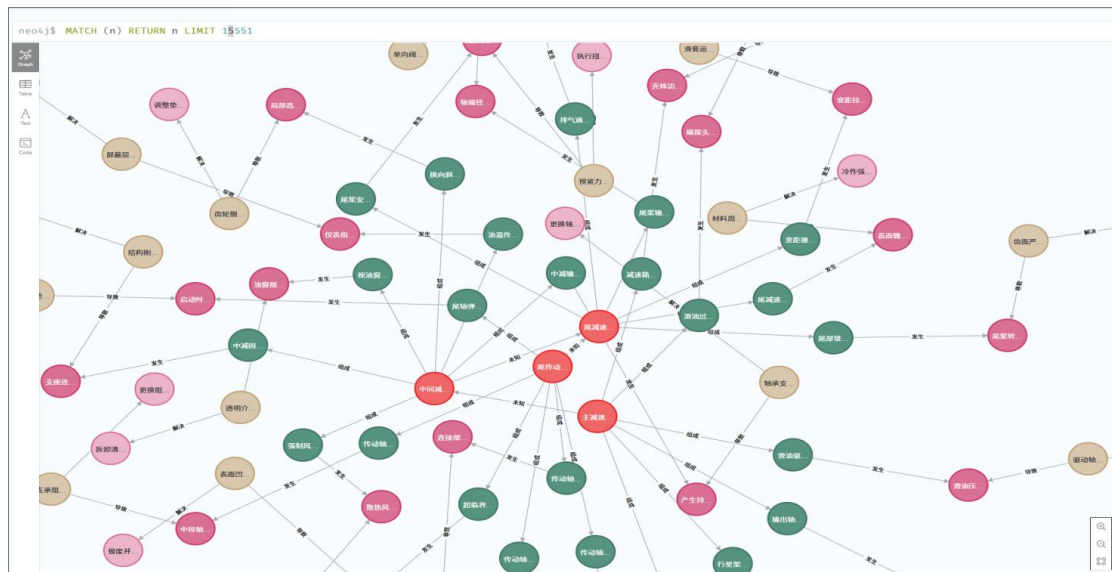


Figure 12. Partial visualization of the helicopter transmission system fault KG

7 Conclusion

An entity–relation joint extraction model integrating reinforcement learning and a GPN is presented to address the challenges of entity overlap and long-range semantic dependencies in helicopter transmission system fault texts. A domain-specific fault dataset for helicopter transmission systems is first constructed based on publicly available corpora. Deep semantic representations are then obtained using BERT, and, for high-density triplet scenarios, an entity-type cross-attention mechanism introduces domain-specific type priors to guide the model in distinguishing the semantic roles of the same entity across different relations, resulting in type-aware enhanced textual representations. To handle long composite entities, a GPN is employed at the decoding stage, enumerating character-span matrices to accurately cover entities exceeding ten characters, thereby avoiding semantic fragmentation and enabling parallel and precise localization of nested entities and non-contiguous overlapping relations. During training, an F1-oriented reinforcement learning optimization framework is adopted to address ambiguous boundaries of domain-specific terminology, using complete expert-annotated triplets as the reward criterion to improve boundary recognition accuracy. Extensive experiments on public benchmarks and the helicopter transmission system fault dataset demonstrate the effectiveness of the proposed model for entity–relation extraction. Finally, the extracted triplets are utilized to complete a preliminary construction of a KG for helicopter transmission system faults.

Data Availability

The data used to support the research findings are available from the corresponding author upon request.

Conflicts of Interest

The authors declare no conflicts of interest.

References

- [1] S. Seo, W. Park, D. Lee, and J. Bae, “Origami-structured actuating modules for upper limb support,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5239–5246, 2021. <https://doi.org/10.1109/LRA.2021.3072810>
- [2] Z. Sun, Q. Zhang, W. Hu, C. Wang, M. Chen, F. Akrami, and C. Li, “A benchmarking study of embedding-based entity alignment for knowledge graphs,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 2326–2340, 2020. <https://doi.org/10.14778/3407790.3407828>
- [3] M. H. Rasmussen, M. Lefrançois, G. F. Schneider, and P. Pauwels, “BOT: The building topology ontology of the W3C linked building data group,” *Semant. Web*, vol. 12, no. 1, pp. 143–161, 2020. <https://doi.org/10.3233/SW-200385>
- [4] P. Chandak, K. Huang, and M. Zitnik, “Building a knowledge graph to enable precision medicine,” *Sci. Data*, vol. 10, no. 1, pp. 67–72, 2023. <https://doi.org/10.1038/s41597-023-01960-3>

- [5] A. Santos, A. R. Colaço, A. B. Nielsen, L. Niu, M. Strauss, P. E. Geyer, F. Coscia, N. J. W. Albrechtsen, F. Mundt, and L. J. Jensen, “A knowledge graph to interpret clinical proteomics data,” *Nat. Biotechnol.*, vol. 40, no. 5, pp. 692–702, 2022. <https://doi.org/10.1038/s41587-021-01145-6>
- [6] O. Kononova, H. Huo, and T. He, “Text-mined dataset of inorganic materials synthesis recipes,” *Sci. Data*, vol. 6, no. 1, pp. 203–213, 2019. <https://doi.org/10.1038/s41597-019-0224-1>
- [7] M. Ammaduddin, S. A. Khan, K. Wennerberg, and T. Aittokallio, “Systematic identification of feature combinations for predicting drug response with Bayesian multi-view multi-task linear regression,” *Bioinformatics*, vol. 33, no. 14, pp. 359–368, 2017. <https://doi.org/10.1093/bioinformatics/btx266>
- [8] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proc. NAACL-HLT*, vol. 37, no. 2, pp. 4171–4186, 2019. <https://doi.org/10.18653/v1/N19-1423>
- [9] F. Li, Z. Wang, S. C. Hui, L. Liao, D. Song, and J. Xu, “Effective named entity recognition with boundary-aware bidirectional neural networks,” *Proc. Web Conf.*, pp. 1695–1703, 2021. <https://doi.org/10.1145/3442381.3449995>
- [10] J. Yu, B. Bohnet, and M. Poesio, “Named entity recognition as dependency parsing,” *Proc. ACL*, pp. 6470–6476, 2020. <https://doi.org/10.18653/v1/2020.acl-main.577>
- [11] J. Akroyd, S. Mosbach, A. Bhave, and M. Kraft, “Universal digital twin: A dynamic knowledge graph,” *Data-Centric Eng.*, vol. 2, p. e14, 2021. <https://doi.org/10.1017/dce.2021.10>
- [12] D. Zeng, K. Liu, Y. Chen, and J. Zhao, “Distant supervision for relation extraction via piecewise convolutional neural networks,” *Proc. EMNLP*, pp. 1753–1762, 2015. <https://doi.org/10.18653/v1/D15-1203>
- [13] M. Miwa and M. Bansal, “End-to-end relation extraction using LSTMs on sequences and tree structures,” *Proc. ACL*, pp. 1105–1116, 2016. <https://doi.org/10.18653/v1/P16-1105>
- [14] J. Hu, J. Gauthier, P. Qian, E. Wilcox, and R. Levy, “A systematic assessment of syntactic generalization in neural language models,” *Proc. ACL*, pp. 1725–1744, 2020. <https://doi.org/10.18653/v1/2020.acl-main.158>
- [15] Y. Wang, B. Yu, Y. Zhang, T. Liu, H. Zhu, and L. Sun, “TPLinker: Single-stage joint extraction of entities and relations through token pair linking,” *Proc. COLING*, pp. 1572–1582, 2020. <https://doi.org/10.18653/v1/2020.coling-main.138>
- [16] A. Ma, Y. Yu, S. Yang, C. Shi, J. Li, and X. Cai, “Survey of knowledge graph based on reinforcement learning,” *J. Comput. Res. Dev.*, vol. 59, no. 8, pp. 1694–1722, 2022. <https://doi.org/10.7544/issn1000-1239.20211264>
- [17] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk, “Scaling deep learning for materials discovery,” *Nature*, vol. 624, pp. 80–85, 2023. <https://doi.org/10.1038/s41586-023-06735-9>
- [18] H. Li, M. Cheng, Z. Yang, L. Yang, and Y. S. Chua, “Named entity recognition for Chinese based on global pointer and adversarial training,” *Sci. Rep.*, vol. 13, no. 1, p. 3242, 2023. <https://doi.org/10.1038/s41598-023-30355-y>
- [19] T. Wang, H. Li, and X. Wang, “Knowledge graph construction based on grid and segment attention mechanism,” *Comput. Integr. Manuf. Syst.*, vol. 31, no. 4, pp. 1368–1382, 2025. <https://doi.org/10.13196/j.cims.2024.0123>
- [20] Z. Zhong and D. Chen, “A frustratingly easy approach for entity and relation extraction,” *arXiv*, p. arXiv:2010.12812, 2021. <https://doi.org/10.48550/arXiv.2010.12812>
- [21] Z. Zhai, R. Fan, J. Huang, and N. Xiong, “A novel joint extraction model based on cross-attention mechanism and global pointer using context shield window,” *Comput. Speech Lang.*, vol. 87, pp. 101 643–101 656, 2024. <https://doi.org/10.1016/j.csl.2024.101643>
- [22] V. Mnih, A. P. Badia, M. Mirza, and A. Graves, “Asynchronous methods for deep reinforcement learning,” *arXiv*, p. arXiv:1602.01783, 2016. <https://doi.org/10.48550/arXiv.1602.01783>
- [23] N. Zhang, Q. Jia, K. Yin, L. Dong, and F. Gao, “Conceptualized representation learning for Chinese biomedical text mining,” *arXiv*, p. arXiv:2008.10813, 2020. <https://doi.org/10.48550/arXiv.2008.10813>
- [24] H. Wei, J. Zhou, Y. Wen, and L. Tang, “Chinese entity relation joint extraction method based on deep learning,” *Comput. Mod.*, no. 8, pp. 10–15, 2025. <https://doi.org/10.3969/j.issn.1006-2475.2025.08.002>